

FORMATION EMBARQUÉE

TP1 — Seance 3

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 1 — Fiche de séance 3 : FSM d'interface, appui long, journalisation (3 h)

En-tête pédagogique

Objectifs	Anti-rebond en pratique ; distinguer appui court/long ; FSM d'interface multi-pages ; min/max ; (option) écrire sur carte SD en tolérant son absence
Prérequis	Séance 2 finie ; TD 03 exercices 2-3
Matériel	Montage séance 2 + bouton poussoir sur D3 (+ module SD SPI en option)
Livrable	Interface 3 pages naviguée au bouton, RAZ min/max par appui long

Déroulé minuté

0:00-0:30 — Module bouton avec appui court / appui long

Nouveau module `bouton.h/.cpp`. Spécification (écris-la d'abord !) : - anti-rebond 20 ms (TD 03) ; - `bouton_evenement()` retourne `AUCUN`, `COURT` (relâché avant 2 s) ou `LONG` (maintenu ≥ 2 s ; l'événement part au franchissement des 2 s, pas au relâchement — meilleure sensation utilisateur).

```
// bouton.h
enum class EvtBouton : uint8_t { AUCUN, COURT, LONG };
void bouton_init();
EvtBouton bouton_tick(uint32_t maintenant_ms);
```

```
// bouton.cpp — FSM à 3 états : RELACHE, APPUYE, LONG_SIGNALE
#include "bouton.h"
#include <Arduino.h>

static const uint8_t BROCHE = 3;
static const uint16_t DEBOUNCE_MS = 20;
static const uint16_t LONG_MS = 2000;

enum class Etat : uint8_t { RELACHE, APPUYE, LONG_SIGNALE };
static Etat etat = Etat::RELACHE;
static bool brut_prec = false;
static uint32_t t_chgt = 0, t_appui = 0;
static bool stable = false;

void bouton_init() { pinMode(BROCHE, INPUT_PULLUP); }

EvtBouton bouton_tick(uint32_t now) {
    bool brut = (digitalRead(BROCHE) == LOW);
    if (brut != brut_prec) { t_chgt = now; brut_prec = brut; }
    if (now - t_chgt > DEBOUNCE_MS) stable = brut;      // état filtré

    switch (etat) {
    case Etat::RELACHE:
        if (stable) { etat = Etat::APPUYE; t_appui = now; }
        break;
    case Etat::APPUYE:
        if (now - t_appui >= LONG_MS) { etat = Etat::LONG_SIGNALE; return EvtBouton::LONG; }
        if (!stable) { etat = Etat::RELACHE; return EvtBouton::COURT; }
        break;
    case Etat::LONG_SIGNALE:           // on attend le relâchement
        if (!stable) etat = Etat::RELACHE; // sans générer d'événement
        break;
    }
    return EvtBouton::AUCUN;
}
```

Remarque de conception : le bouton est LUI-MÊME une petite FSM — dessine-la dans ton journal (3 états, 4 transitions). **Test** : `Serial.println` de chaque événement ; vérifie qu'un appui long ne génère PAS un court au relâchement (c'est le rôle de `LONG_SIGNALE`).

0:30-1:00 — Min/max dans le module capteur

Ajoute à `capteur.h` : `capteur_temp_min()`, `capteur_temp_max()`, `capteur_raz_minmax()`.

Implémentation : mise à jour à chaque mesure valide (attention à l'initialisation : partir de la première mesure, pas de 0 — un min initialisé à 0 est faux dans une pièce à 20 °C ; utilise NAN comme sentinelle).

1:00-2:00 — La FSM de pages

Dans le `.ino` (c'est de l'orchestration, donc c'est sa place) :

```
enum class Page : uint8_t { MESURES, MINMAX, CONSIGNE };
Page page = Page::MESURES;

// dans loop() :
EvtBouton evt = bouton_tick(now);
if (evt == EvtBouton::COURT) {
    page = (page == Page::MESURES) ? Page::MINMAX
        : (page == Page::MINMAX) ? Page::CONSIGNE
        : Page::MESURES;
}
if (evt == EvtBouton::LONG && page == Page::MINMAX) {
    capteur_raz_minmax(); // RAZ seulement depuis la page concernée
}
```

Et `affichage_tick` prend maintenant la page en paramètre et dessine la bonne vue (3 fonctions statiques internes : `dessiner_mesures()`, `dessiner_minmax()`, `dessiner_consigne()`).

✓ **Point de contrôle (2:00)** : navigation fluide pendant que les mesures continuent ; appui long sur la page MIN/MAX → valeurs remises à la mesure courante ; appui long ailleurs → rien (comportement **spécifié**, pas accidentel).

2:00-2:50 — (option matériel) Journalisation SD

Module `journal.h/.cpp` avec la carte SD (CS=D10, bibliothèque `SD`) : - toutes les 60 s : ligne CSV `millis;temperature;humidite` dans `METEO.CSV` ; - **l'absence de carte est un état normal** : `journal_ok()` renvoie false, l'écran ajoute un pictogramme, et le module retente l'init toutes les 30 s. Rien ne bloque, rien ne plante.

Sans matériel SD : écris le même module mais vers `Serial` (préfixe `CSV;`) — l'architecture est le vrai objectif.

2:50-3:00 — Commit + journal de bord

```
git add . && git commit -m "TP1 seance 3 : FSM pages, appui long, journal"
```

Erreurs fréquentes

Symptôme	Cause	Remède
Un appui = 2 ou 3 changements de page	anti-rebond absent/trop court, ou événement généré sur niveau au lieu de front	revoir la FSM bouton
L'appui long déclenche aussi un court	pas d'état <code>LONG_SIGNALE</code>	voir le corrigé ci-dessus
min/max absurdes (0 ou -273)	initialisation à 0 au lieu de la 1 ^{re} mesure	sentinelle NAN
Init SD qui gèle 2 s toute la boucle	<code>SD.begin()</code> est bloquant	ne le retenter que toutes les 30 s, hors du chemin chaud
Plus de RAM (Uno)	SD + OLED + chaînes	<code>F()</code> partout ; c'est le moment de mesurer avec l'IDE

Travail à la maison (30 min)

Sur papier : dessine le **diagramme d'états complet** de ton application (pages × événements) et la FSM du bouton. Range les deux dessins dans le dépôt (photo `docs/fsm.jpg`) — un firmware dont la FSM n'est pas dessinée n'est pas fini.

➡ Fiche suivante : [Séance 4 — ESP32 et MQTT](#)